Unit-2

Conditional Statement Programs

Program 1

Aim

To write a Python program to check whether a given number is positive, negative, or zero.

Algorithm

1. Start.
2. Read a number from the user.
3. If the number is greater than 0, display "Positive".
4. If the number is less than 0, display "Negative".
5. Otherwise, display "Zero".
6. Stop.

Python Program

```
num = float(input("Enter a number: "))

if num > 0:
   print("The number is Positive")
elif num < 0:
   print("The number is Negative")
else:
   print("The number is Zero")
```

Output

```
Enter a number: -8
The number is Negative
```

Result

Thus, the program successfully checks whether a number is positive, negative, or zero.

Program 2

Aim

To write a Python program to find the largest of three numbers using nested if statements.

Algorithm

1. Start.
2. Read three numbers.
3. Compare the first number with the second number.
4. Using nested if, compare the larger number with the third number.
5. Display the largest number.
6. Stop.

Python Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a > b:
    if a > c:
        largest = a
    else:
        largest = c
else:
    if b > c:
        largest = b
    else:
        largest = c

print("Largest number is:", largest)
```

Output

```
Enter first number: 25
Enter second number: 48
Enter third number: 32
Largest number is: 48
```

Result

Thus, the program successfully finds the largest of three numbers using nested if statements.

Program 3

Aim

To write a Python program to find the smallest of three numbers using nested if statements.

Algorithm

1. Start.
2. Read three numbers.
3. Compare the first number with the second number.
4. Using nested if, compare the smaller number with the third number.
5. Display the smallest number.
6. Stop.

Python Program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a < b:
    if a < c:
        smallest = a
    else:
        smallest = c
else:
    if b < c:
        smallest = b
    else:
        smallest = c

print("Smallest number is:", smallest)
```

Output

Enter first number: 25
Enter second number: 48
Enter third number: 12
Smallest number is: 12

Result

Thus, the program successfully finds the smallest of three numbers using nested if statements.

Program 4

Aim

To write a Python program to check whether a given year is a leap year.

Algorithm

1. **Start**
2. Read the year.
3. Check whether the year is divisible by 4.
4. If yes, use a nested if to check whether the year is divisible by 100.
5. If divisible by 100, use another nested if to check whether it is divisible by 400.
6. If divisible by 400, display **Leap Year**.
7. Otherwise, display **Not a Leap Year**.
8. If the year is not divisible by 100, display **Leap Year**.
9. If the year is not divisible by 4, display **Not a Leap Year**.
10. **Stop**

Python Program

```
year = int(input("Enter a year: "))

if year % 4 == 0:
   if year % 100 == 0:
      if year % 400 == 0:
         print("Leap Year")
      else:
         print("Not a Leap Year")
   else:
      print("Leap Year")
else:
   print("Not a Leap Year")
```

Output

Enter a year: 2024
The year is a Leap Year

Result

Thus, the program successfully checks whether a given year is a leap year.

Program 5

Aim

To write a Python program to check whether a person is eligible to vote using if-else.

Algorithm

1. Start.
2. Read the person's age.
3. If the age is 18 or above, display "Eligible to Vote".
4. Otherwise, display "Not Eligible to Vote".
5. Stop.

Python Program

```
age = int(input("Enter your age: "))

if age >= 18:
   print("You are eligible to vote")
else:
   print("You are not eligible to vote")
```

Output

Enter your age: 20
You are eligible to vote

Result

Thus, the program successfully checks voting eligibility based on age.

Program 6
Aim
To write a Python program to assign a grade to a student based on marks using an elif ladder.

Algorithm
1. Start.
2. Read the marks obtained by the student.
3. Check the marks using an if-elif-else ladder.
4. Assign the appropriate grade.
5. Display the grade.
6. Stop.

Python Program
```
marks = float(input("Enter marks: "))

if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 50:
    grade = "D"
else:
    grade = "F"

print("Grade:", grade)
```

Output
Enter marks: 85
Grade: A

Result
Thus, the program successfully assigns a grade to a student using an elif ladder.

Program 7
Aim
To write a Python program to check whether a given number is prime.

Algorithm
1. **Start**
2. Read a number num from the user.

3. Check whether num > 1.
4. If num > 1, repeat the following steps for every value of i from 2 to num - 1:
    - Check whether num % i == 0.
    - If the remainder is 0, display **"num is not a prime number"** and stop the loop.
5. If the loop completes without finding any divisor, display **"num is a prime number"**.
6. If num <= 1, display **"num is not a prime number"**.
7. **Stop**.

Python Program

```
num = int(input("Enter a number: "))
if num > 1:
   for i in range(2,num):
     if (num % i) == 0:
        print(num,"is not a prime number")
        break
  else:
     print(num,"is a prime number")

else:
  print(num,"is not a prime number")
```

Output

Enter a number: 29
The number is a Prime Number

Result

Thus, the program successfully checks whether a given number is prime.

Program 8

Aim

To write a Python program to check whether a triangle is valid based on its three angles.

Algorithm

1. Start.
2. Read the three angles of the triangle.
3. Check whether all angles are greater than 0.
4. Check whether the sum of the three angles is equal to 180°.
5. If both conditions are satisfied, the triangle is valid.
6. Otherwise, the triangle is invalid.
7. Stop.

Python Program

```
angle1 = float(input("Enter first angle: "))
angle2 = float(input("Enter second angle: "))
angle3 = float(input("Enter third angle: "))
```

```
if angle1 > 0 and angle2 > 0 and angle3 > 0 and angle1 + angle2 + angle3 == 180:
    print("The triangle is Valid")
else:
    print("The triangle is Invalid")
```

Output

```
Enter first angle: 60
Enter second angle: 70
Enter third angle: 50
The triangle is Valid
```

Result

Thus, the program successfully checks whether a triangle is valid based on its three angles.

Iterative Statement Programs

Program 9

Aim

To write a Python program to print numbers from 1 to N using a for loop.

Algorithm

1. Start.
2. Read the value of N.
3. Use a for loop from 1 to N.
4. Print each number.
5. Stop.

Python Program

```
n = int(input("Enter the value of N: "))

for i in range(1, n + 1):
    print(i)
```

Output

```
Enter the value of N: 5
1
2
3
4
5
```

Result

Thus, the program successfully prints numbers from 1 to N using a for loop.

Program 10
Aim
To write a Python program to find the sum of digits of a given number.

Algorithm
1. Start.
2. Read a number.
3. Initialize sum = 0.
4. Extract the last digit using the modulus operator %.
5. Add the digit to sum.
6. Remove the last digit using integer division //.
7. Repeat until the number becomes 0.
8. Display the sum.
9. Stop.

Python Program
```
num = int(input("Enter a number: "))

sum = 0

while num > 0:
    digit = num % 10
    sum = sum + digit
    num = num // 10

print("Sum of digits =", sum)
```

Output
Enter a number: 12345
Sum of digits = 15

Result
Thus, the program successfully finds the sum of digits of a given number.

Program 11
Aim
To write a Python program to reverse a given number.

Algorithm
1. Start.
2. Read a number.
3. Initialize reverse = 0.
4. Extract the last digit of the number.
5. Add the digit to the reverse number.
6. Remove the last digit from the original number.
7. Repeat until the number becomes 0.

8. Display the reversed number.
9. Stop.

Python Program

```
num = int(input("Enter a number: "))

reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10

print("Reversed number =", reverse)
```

Output

```
Enter a number: 12345
Reversed number = 54321
```

Result

Thus, the program successfully reverses a given number.

Program 12

Aim

To write a Python program to print the multiplication table of a given number.

Algorithm

1. Start.
2. Read a number.
3. Use a for loop from 1 to 10.
4. Multiply the number by the loop variable.
5. Display the result.
6. Stop.

Python Program

```
num = int(input("Enter a number: "))

for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Output

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
```

5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

Result
Thus, the program successfully prints the multiplication table of a given number.

Program 13
Aim
To write a Python program to find the factorial of a given number.

Algorithm
1. Start.
2. Read a number n.
3. Initialize factorial = 1.
4. Use a for loop from 1 to n.
5. Multiply factorial by each number.
6. Display the factorial.
7. Stop.

Python Program
```
n = int(input("Enter a number: "))

factorial = 1

for i in range(1, n + 1):
    factorial = factorial * i

print("Factorial =", factorial)
```

Output
Enter a number: 5
Factorial = 120

Result
Thus, the program successfully calculates the factorial of a given number.

Program 14
Aim
To write a Python program to print all prime numbers between 1 and N.

Algorithm

1. Start.
2. Read the value of N.
3. Start checking numbers from 2 to N.
4. For each number, assume it is prime.
5. Check whether it is divisible by any number from 2 to one less than itself.
6. If divisible, mark it as not prime.
7. If it remains prime, print the number.
8. Repeat until N.
9. Stop.

Python Program

```
n = int(input("Enter the value of N: "))

print("Prime numbers between 1 and", n, "are:")

for num in range(2, n + 1):
   for i in range(2, num):
      if num % i == 0:
         break
   else:
      print(num)
```

Output

```
Enter the value of N: 20
Prime numbers between 1 and 20 are:
2
3
5
7
11
13
17
19
```

Result

Thus, the program successfully prints all prime numbers between 1 and N.

Program 15

Aim

To write a Python program to find the largest number in a series of N numbers entered by the user.

Algorithm

1. Start.
2. Read the number of values N.

3. Read the first number and assume it is the largest.
4. Read the remaining numbers one by one.
5. Compare each number with the current largest number.
6. If the number is greater, update the largest number.
7. Display the largest number.
8. Stop.

Python Program

```
n = int(input("Enter the number of values: "))

largest = int(input("Enter number 1: "))

for i in range(2, n + 1):
    num = int(input("Enter number " + str(i) + ": "))

    if num > largest:
        largest = num

print("Largest number =", largest)
```

Output

```
Enter the number of values: 5
Enter number 1: 12
Enter number 2: 45
Enter number 3: 23
Enter number 4: 67
Enter number 5: 34
Largest number = 67
```

Result

Thus, the program successfully finds the largest number in a series of N numbers.

Program 16

Aim

To write a Python program to find the sum and average of N numbers.

Algorithm

1. Start.
2. Read the number of values N.
3. Initialize sum = 0.
4. Read N numbers using a loop.
5. Add each number to sum.
6. Calculate average using sum / N.
7. Display the sum and average.
8. Stop.

Python Program

```
n = int(input("Enter the number of values: "))

sum = 0

for i in range(1, n + 1):
    num = float(input("Enter number " + str(i) + ": "))
    sum = sum + num

average = sum / n

print("Sum =", sum)
print("Average =", average)
```

Output

```
Enter the number of values: 4
Enter number 1: 10
Enter number 2: 20
Enter number 3: 30
Enter number 4: 40
Sum = 100.0
Average = 25.0
```

Result

Thus, the program successfully calculates the sum and average of N numbers.

Program 17

Aim

To write a Python program to print the Fibonacci series up to N terms.

Algorithm

1. Start.
2. Read the number of terms N.
3. Initialize the first two terms as 0 and 1.
4. Print the current term.
5. Calculate the next term by adding the previous two terms.
6. Repeat until N terms are printed.
7. Stop.

Python Program

```
n = int(input("Enter the number of terms: "))

a = 0
b = 1
```

```
print("Fibonacci Series:")

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output

Enter the number of terms: 8
Fibonacci Series:
0 1 1 2 3 5 8 13

Result

Thus, the program successfully prints the Fibonacci series up to N terms.

Program 18

Aim

To write a Python program to check whether a given number is an Armstrong number.

Algorithm

1. Start.
2. Read a number.
3. Store the original number in a temporary variable.
4. Count the number of digits.
5. Extract each digit.
6. Find the sum of each digit raised to the power of the number of digits.
7. Compare the calculated sum with the original number.
8. If they are equal, display "Armstrong Number".
9. Otherwise, display "Not an Armstrong Number".
10. Stop.

Python Program

```
num = int(input("Enter a number: "))

original = num
digits = len(str(num))
sum = 0

while num > 0:
    digit = num % 10
    sum = sum + digit ** digits
    num = num // 10

if sum == original:
    print(original, "is an Armstrong number")
else:
    print(original, "is not an Armstrong number")
```

Output
Enter a number: 153
153 is an Armstrong number

Result
Thus, the program successfully checks whether a given number is an Armstrong number.

Program 19
Aim
To write a Python program to check whether a given number is a palindrome.

Algorithm
1. Start.
2. Read a number.
3. Store the original number in a temporary variable.
4. Initialize reverse = 0.
5. Extract the last digit of the number.
6. Construct the reverse of the number.
7. Compare the reversed number with the original number.
8. If both are equal, display "Palindrome".
9. Otherwise, display "Not a Palindrome".
10. Stop.

Python Program
```
num = int(input("Enter a number: "))

original = num
reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10

if reverse == original:
    print(original, "is a palindrome")
else:
    print(original, "is not a palindrome")
```

Output
Enter a number: 121
121 is a palindrome

Result
Thus, the program successfully checks whether a given number is a palindrome.

Program 20
Aim
To write a Python program to display the multiplication table of a given number using a while loop.

Algorithm
1. Start.
2. Read a number from the user.
3. Initialize i = 1.
4. Repeat while i <= 10.
5. Multiply the given number by i.
6. Display the result.
7. Increment i by 1.
8. Stop.

Python Program
```
num = int(input("Enter a number: "))

i = 1

while i <= 10:
    print(num, "x", i, "=", num * i)
    i = i + 1
```

Output
```
Enter a number: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Result
Thus, the program successfully displays the multiplication table using a while loop.

Program 21
Aim
To write a Python program to count the number of digits in a given number.

Algorithm

1. Start.
2. Read a number.
3. Initialize count = 0.
4. Repeatedly divide the number by 10 using integer division.
5. Increment count for every division.
6. Repeat until the number becomes 0.
7. Display the number of digits.
8. Stop.

Python Program

```
num = int(input("Enter a number: "))

count = 0

while num > 0:

    digit = num % 10
    count+ = 1
    num = num // 10

print(“Number of digits =”,count)
```

Output

Enter a number: 12345
Number of digits = 5

Result

Thus, the program successfully counts the number of digits in a given number.

Program 22

Aim

To write a Python program to demonstrate the use of the break statement in a loop.

Algorithm

1. Start.
2. Use a for loop to print numbers from 1 to 10.
3. Check whether the current number is 5.
4. If the number is 5, use the break statement.
5. The loop terminates immediately.
6. Stop.

Python Program

```
for i in range(1, 11):
   if i == 5:
      break
```

```
    print(i)

print("Loop terminated using break")
```

Output

```
1
2
3
4
Loop terminated using break
```

Result

Thus, the program successfully demonstrates the use of the break statement to terminate a loop.

Program 23

Aim

To write a Python program to demonstrate the use of the continue statement in a loop.

Algorithm

1. Start.
2. Use a for loop to print numbers from 1 to 10.
3. Check whether the current number is 5.
4. If the number is 5, use the continue statement.
5. The current iteration is skipped.
6. Continue with the next iteration.
7. Stop.

Python Program

```
for i in range(1, 11):
    if i == 5:
        continue
    print(i)
```

Output

```
1
2
3
4
6
7
8
9
10
```

Result
Thus, the program successfully demonstrates the use of the continue statement to skip an iteration of a loop.

Program 24
Aim
To write a Python program to demonstrate the use of the pass statement in a loop.

Algorithm
1. Start.
2. Use a for loop to iterate from 1 to 5.
3. Check whether the current number is 3.
4. If the number is 3, execute the pass statement.
5. The pass statement does nothing and the loop continues normally.
6. Print the numbers.
7. Stop.

Python Program
```
for i in range(1, 6):
    if i == 3:
        pass
    print(i)
```

Output
```
1
2
3
4
5
```

Result
Thus, the program successfully demonstrates the use of the pass statement in a loop.

Pattern Generation Programs

Program 25
Aim
To write a Python program to print a right-angled triangle pattern using stars.

Algorithm
1. Start.
2. Read the number of rows.
3. Use an outer for loop to control the rows.
4. Use an inner for loop to print stars.
5. Print the required number of stars in each row.
6. Move to the next line after each row.

7. Stop.
8.

Python Program

```
rows = int(input("Enter the number of rows: "))

for i in range(1, rows + 1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output

```
Enter the number of rows: 5
*
* *
* * *
* * * *
* * * * *
```

Result

Thus, the program successfully prints a right-angled triangle pattern of stars.

Program 26

Aim

To write a Python program to print an inverted right-angled triangle pattern using stars.

Algorithm

1. Start.
2. Read the number of rows.
3. Use an outer for loop starting from the given number of rows.
4. Use an inner for loop to print stars.
5. Decrease the number of stars in each successive row.
6. Stop.

Python Program

```
rows = int(input("Enter the number of rows: "))

for i in range(rows, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output

```
Enter the number of rows: 5
* * * * *
* * * *
* * *
```

* *
*

Result
Thus, the program successfully prints an inverted right-angled triangle pattern.

Program 27
Aim
To write a Python program to print a pyramid pattern of numbers.

Algorithm
1. Start.
2. Read the number of rows.
3. Use an outer loop to control the rows.
4. Print spaces before the numbers to create the pyramid shape.
5. Use another loop to print numbers in each row.
6. Print the required numbers.
7. Move to the next row.
8. Stop.

Python Program
```
rows = int(input("Enter the number of rows: "))

for i in range(1, rows + 1):
    for j in range(rows - i):
        print(" ", end=" ")

    for j in range(1, i + 1):
        print(j, end=" ")

    print()
```

Output
```
Enter the number of rows: 5
        1
      1 2
    1 2 3
  1 2 3 4
1 2 3 4 5
```

Result
Thus, the program successfully prints a pyramid pattern of numbers.

Program 28

Aim

To write a Python program to print a diamond pattern using stars.

Algorithm

1. Start.
2. Read the number of rows for the upper half.
3. Print the upper half of the diamond using spaces and stars.
4. Print the lower half using decreasing numbers of stars.
5. Display the complete diamond pattern.
6. Stop.

Python Program

```
rows = int(input("Enter the number of rows: "))

# Upper half
for i in range(1, rows + 1):
    for j in range(rows - i):
        print(" ", end=" ")

    for j in range(2 * i - 1):
        print("*", end=" ")

    print()

# Lower half
for i in range(rows - 1, 0, -1):
    for j in range(rows - i):
        print(" ", end=" ")

    for j in range(2 * i - 1):
        print("*", end=" ")

    print()
```

Output

```
Enter the number of rows: 4
     *
    * * *
  * * * * *
* * * * * * *
  * * * * *
    * * *
     *
```

Result
Thus, the program successfully prints a diamond pattern of stars.

Program 29
Aim
To write a Python program to print Pascal's triangle.

Algorithm
1. Start.
2. Read the number of rows.
3. Initialize the first value of each row as 1.
4. Use nested loops to generate each row.
5. Calculate each value using the previous values.
6. Print the values in a triangular format.
7. Stop.

Python Program
```
rows = int(input("Enter the number of rows: "))

for i in range(rows):
    print(" " * (rows - i), end="")

    value = 1

    for j in range(i + 1):
        print(value, end=" ")
        value = value * (i - j) // (j + 1)

    print()
```

Output
```
Enter the number of rows: 5
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

Result
Thus, the program successfully prints Pascal's triangle.

Program 30
Aim
To write a Python program to print Floyd's triangle.

Algorithm

1. Start.
2. Read the number of rows.
3. Initialize a number variable to 1.
4. Use an outer loop for the rows.
5. Use an inner loop to print numbers in each row.
6. Increment the number after printing each value.
7. Continue until all rows are printed.
8. Stop.

Python Program

```
rows = int(input("Enter the number of rows: "))

num = 1

for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(num, end=" ")
        num = num + 1
    print()
```

Output

```
Enter the number of rows: 5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Result

Thus, the program successfully prints Floyd's triangle.

String Manipulation Programs

Program 31

Aim

To write a Python program to demonstrate string creation and basic string operations such as concatenation, repetition, and slicing.

Algorithm

1. Start.
2. Create two strings.
3. Perform concatenation using the + operator.
4. Perform repetition using the * operator.
5. Perform slicing using the slicing operator [:].
6. Display the results.

7. Stop.

Python Program

```
str1 = "Hello"
str2 = "World"

print("String 1:", str1)
print("String 2:", str2)

# Concatenation
print("Concatenation:", str1 + " " + str2)

# Repetition
print("Repetition:", str1 * 3)

# Slicing
print("Slicing:", str2[0:3])
```

Output

String 1: Hello
String 2: World
Concatenation: Hello World
Repetition: HelloHelloHello
Slicing: Wor

Result

Thus, the program successfully demonstrates string creation, concatenation, repetition, and slicing.

Program 32

Aim

To write a Python program to demonstrate commonly used string methods such as upper(), lower(), strip(), split(), join(), replace(), and find().

Algorithm

1. Start.
2. Create a string containing spaces and words.
3. Apply the upper() method.
4. Apply the lower() method.
5. Remove extra spaces using strip().
6. Divide the string using split().
7. Join words using join().
8. Replace a word using replace().
9. Search for a substring using find().
10. Display all results.
11. Stop.

Python Program

```
text = "  Hello Python World  "

print("Original String:", text)

print("Upper:", text.upper())
print("Lower:", text.lower())
print("Strip:", text.strip())

words = text.strip().split()
print("Split:", words)

print("Join:", "-".join(words))

print("Replace:", text.replace("Python", "Programming"))

print("Find:", text.find("Python"))
```

Output

```
Original String:   Hello Python World
Upper:   HELLO PYTHON WORLD
Lower:   hello python world
Strip: Hello Python World
Split: ['Hello', 'Python', 'World']
Join: Hello-Python-World
Replace:   Hello Programming World
Find: 8
```

Result

Thus, the program successfully demonstrates various built-in string methods.

Program 33

Aim

To write a Python program to check whether a given string is a palindrome.

Algorithm

1. Start.
2. Read a string from the user.
3. Reverse the string using slicing.
4. Compare the original string with its reverse.
5. If both are equal, display "Palindrome".
6. Otherwise, display "Not a Palindrome".
7. Stop.

Python Program

```
text = input("Enter a string: ")

reverse = text[::-1]

if text == reverse:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")
```

Output

Enter a string: madam
The string is a palindrome

Result

Thus, the program successfully checks whether a given string is a palindrome.

Program 34

Aim

To write a Python program to reverse a given string.

Algorithm

1. Start.
2. Read a string from the user.
3. Use slicing with a step of -1.
4. Store the reversed string.
5. Display the reversed string.
6. Stop.

Python Program

```
text = input("Enter a string: ")

reverse = text[::-1]

print("Reversed string:", reverse)
```

Output

Enter a string: Python
Reversed string: nohtyP

Result

Thus, the program successfully reverses a given string.

Program 35

Aim

To write a Python program to count the number of vowels, consonants, digits, and spaces in a given string.

Algorithm

1. Start.
2. Read a string.
3. Initialize counters for vowels, consonants, digits, and spaces to zero.
4. Examine each character in the string.
5. If the character is a vowel, increment the vowel counter.
6. If the character is an alphabet but not a vowel, increment the consonant counter.
7. If the character is a digit, increment the digit counter.
8. If the character is a space, increment the space counter.
9. Display all counts.
10. Stop.

Python Program

```
text = input("Enter a string: ")

vowels = 0
consonants = 0
digits = 0
spaces = 0

for char in text:
    if char.lower() in "aeiou":
        vowels += 1
    elif char.isalpha():
        consonants += 1
    elif char.isdigit():
        digits += 1
    elif char == " ":
        spaces += 1

print("Vowels:", vowels)
print("Consonants:", consonants)
print("Digits:", digits)
print("Spaces:", spaces)
```

Output

```
Enter a string: Hello Python 123
Vowels: 3
Consonants: 8
Digits: 3
Spaces: 2
```

Result
Thus, the program successfully counts vowels, consonants, digits, and spaces in a string.

Program 36
Aim
To write a Python program to demonstrate string formatting using format(), f-strings, and the % operator.

Algorithm
1. Start.
2. Create variables containing a name and age.
3. Format the values using the format() method.
4. Format the values using an f-string.
5. Format the values using the % operator.
6. Display all three formatted strings.
7. Stop.

Python Program
```
name = "Rahul"
age = 20

# Using format()
print("Using format(): My name is {} and I am {} years old.".format(name, age))

# Using f-string
print(f"Using f-string: My name is {name} and I am {age} years old.")

# Using % operator
print("Using % operator: My name is %s and I am %d years old." % (name, age))
```

Output
Using format(): My name is Rahul and I am 20 years old.
Using f-string: My name is Rahul and I am 20 years old.
Using % operator: My name is Rahul and I am 20 years old.

Result
Thus, the program successfully demonstrates three different methods of string formatting in Python.

Program 37
Aim
To write a Python program to count the frequency of each character in a given string.

Algorithm
1. Start.
2. Read a string.
3. Create an empty dictionary.
4. Traverse each character in the string.
5. If the character is already present in the dictionary, increase its count.
6. Otherwise, add the character with a count of 1.
7. Display the frequency of each character.
8. Stop.

Python Program
```
text = input("Enter a string: ")

frequency = {}

for char in text:
    if char in frequency:
        frequency[char] += 1
    else:
        frequency[char] = 1

print("Character frequencies:")

for char, count in frequency.items():
    print(char, ":", count)
```

Output
```
Enter a string: hello
Character frequencies:
h : 1
e : 1
l : 2
o : 1
```

Result
Thus, the program successfully calculates the frequency of each character in a string.

Program 38

Aim

To write a Python program to check whether two given strings are anagrams.

Algorithm

1. Start.
2. Read two strings.
3. Convert both strings to lowercase.
4. Remove spaces from both strings.
5. Sort the characters of both strings.
6. Compare the sorted strings.
7. If they are equal, display "Anagrams".
8. Otherwise, display "Not Anagrams".
9. Stop.

Python Program

```
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")

str1 = str1.replace(" ", "").lower()
str2 = str2.replace(" ", "").lower()

if sorted(str1) == sorted(str2):
   print("The strings are anagrams")
else:
   print("The strings are not anagrams")
```

Output

```
Enter first string: listen
Enter second string: silent
The strings are anagrams
```

Result

Thus, the program successfully checks whether two given strings are anagrams.